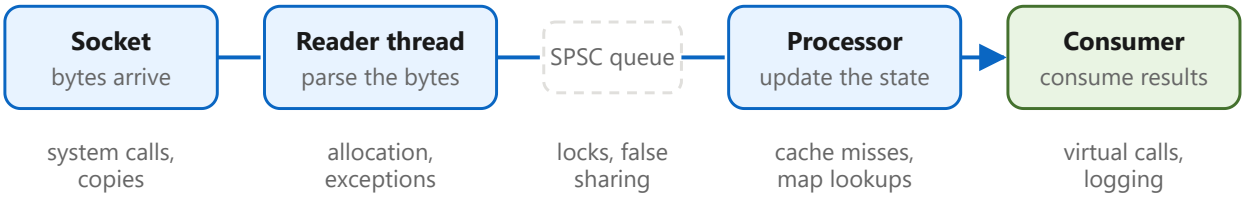
Performance and Memory

How a message-processing program keeps its hot path fast and *predictable*: measuring tail latency, avoiding allocation, cache-friendly data, dense-key tables, queues between threads, and pinning threads to cores. Compiled examples are in `examples/lessons/12-low-latency`. Assumes Multithreading and Binary Data and Parsing.

Where latency comes from



Every stage must be fast AND predictable.

One occasional 50 microsecond stall anywhere shows up at the end of the pipeline.

A hot path is a chain; the slowest rare event in any link is what you feel.

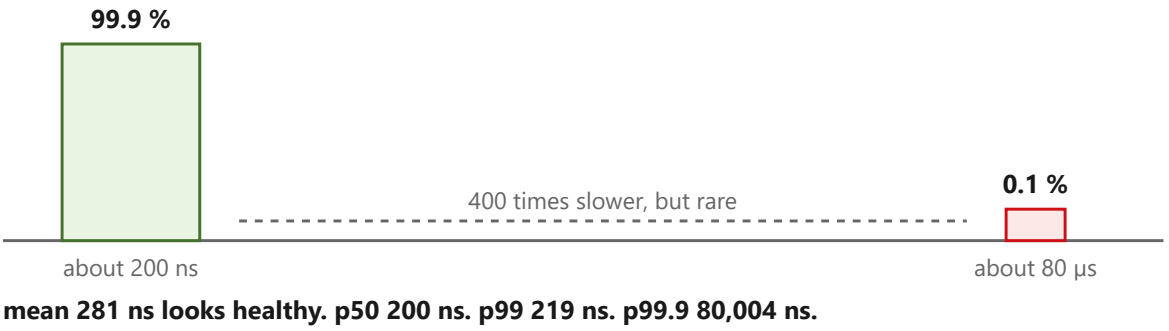
- **Latency** is the time for one message to get through; **throughput** is how many per second. A system can have great throughput and terrible latency.
- In a message pipeline the **tail** matters: the rare slow case (once in a thousand) is what causes a stale result or a missed deadline. Average time hides it.
- So the goal is not only "fast" but **predictable**: remove anything whose cost can suddenly jump.

SOURCE	ROUGH COST (ORDERS OF MAGNITUDE)	REMEDY
L1 cache hit versus main memory	about 1 ns versus about 100 ns	contiguous data, small hot structs
Heap allocation (`new`)	tens of ns, but unbounded in the worst case (locks, page faults)	preallocate, pools, arenas
Mutex contention, context switch	microseconds	single-writer queues, pinning

System call (read, write, log)	hundreds of ns to microseconds	batch, keep off the hot path
Page fault	microseconds	pre-touch memory at startup
Throwing an exception	microseconds	error codes, `optional`

The numbers vary a lot by CPU and system; treat them as ratios. The rule that matters: **measure first**, then remove the biggest source.

Measuring: percentiles and tail latency



mean 281 ns looks healthy. p50 200 ns. p99 219 ns. p99.9 80,004 ns.

The average hides the tail. (Numbers from the program in 12-low-latency.)

- **Never judge by the average.** Report **percentiles**: p50 (median), p99, p99.9, and the maximum. p99.9 means 1 in 1000 operations is at least that slow.
- Collect every sample (do not average on the fly), sort afterwards, read the percentile by rank.

```cpp
std::uint64_t percentile(const std::vector<std::uint64_t>& sorted, double p)
{
    std::size_t rank = static_cast<std::size_t>(p / 100.0 * sorted.size());
    return sorted[std::min(rank, sorted.size() - 1)];
}

auto t0 = std::chrono::steady_clock::now();          // monotonic: never goes backwards
doWork();
auto t1 = std::chrono::steady_clock::now();
samples.push_back(duration_cast<nanoseconds>(t1 - t0).count());   // reserve() the vector first
```

Measuring pitfalls

- **The compiler deletes unused work.** If a benchmark's result is never used, the loop may be optimized away. Store results into a `volatile` sink, or print a checksum.
- **Warm up.** The first iterations pay for cold caches and page faults. Discard them, or report them separately.
- **Do not disturb what you measure:** no printing or allocation inside the timed region, and the clock call itself costs tens of nanoseconds.
- Measure in realistic conditions (real message rates, other threads running). A quiet benchmark hides the stalls that matter.

Memory on the hot path

`new` and `malloc` can take a lock, walk free lists, call into the kernel for more memory and fragment the heap. Usually fast, but occasionally slow and **unpredictable**. The rule: **allocate everything at startup; the hot path never calls `new`**.

- **Preallocate containers:** `reserve()` a vector so it never reallocates (one allocation instead of several as it grows).
- **Object pool:** a fixed array of slots plus a **free list**. Allocation and release are a few instructions; if the pool is exhausted, you decide what to do (drop, count, wait).
- **Arena (`std::pmr::monotonic_buffer_resource`):** allocation is "bump a pointer" inside a buffer you provide; everything is released at once.
- Also: `std::array` and fixed buffers instead of `std::string` and `std::vector` for small data, and no `std::function` capturing large state (it may allocate).

Step through a pool:

STEP THROUGH **An object pool with a free list (4 slots)**

```
1  Pool<Task, 4> pool;
2  Task* a = pool.allocate(1, cost, 300);
3  Task* b = pool.allocate(2, cost, 100);
4  pool.deallocate(a);
5  Task* c = pool.allocate(3, cost, 50);
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 1 `Pool<Task, 4> pool;`	At startup every slot is **free** and the slots are linked into a **free list**: slot0 to slot1 to slot2 to slot3. **freeHead** points at slot0. All the memory exists already; nothing will be allocated later.	**freeHead = 0** | **slot0 = next = 1** | **slot1 = next = 2** | **slot2 = next = 3** | **slot3 = next = 4 (end)**
2	line 2 `Task* a = pool.allocate(1, cost, 300);`	`allocate` **pops the list**: it takes slot0, moves **freeHead** to slot1, and constructs the Task inside slot0 with **placement new**. A handful of instructions, no `new`.	**freeHead = 1** | **a = slot0** | **slot0 = Task #1** | slot1 = next = 2 | slot2 = next = 3 | slot3 = next = 4 (end)
3	line 3 `Task* b = pool.allocate(2, cost, 100);`	The second allocation takes slot1. Two slots in use, **freeHead** now at slot2.	**freeHead = 2** | a = slot0 | **b = slot1** | slot0 = Task #1 | **slot1 = Task #2** | slot2 = next = 3 | slot3 = next = 4 (end)
4	line 4 `pool.deallocate(a);`	`deallocate(a)` runs the destructor and **pushes slot0 on the front of the free list**: slot0 links to slot2 and **freeHead** points at slot0. The pointer **a** is dangling now, as after any free.	**freeHead = 0** ~~| a = dangling~~ | b = slot1 | **slot0 = next = 2** | slot1 = Task #2 | slot2 = next = 3 | slot3 = next = 4 (end)
5	line 5 `Task* c = pool.allocate(3, cost, 50);`	The next allocation reuses slot0, the slot just freed: warm in the cache and at the same address as `a` had. When all four slots are in use, `allocate` returns `nullptr` and the caller chooses (drop, count, wait). Constant time, no heap, no fragmentation.	**freeHead = 2** | b = slot1 | **c = slot0** | **slot0 = Task #3** | slot1 = Task #2 | slot2 = next = 3 | slot3 = next = 4 (end)

```
template <typename T, std::size_t N>
class Pool {
    union Slot { alignas(T) unsigned char storage[sizeof(T)]; std::size_t
nextFree; };
    std::array<Slot, N> slots_;
    std::size_t freeHead_ = 0;                          // first free slot; N
means none
public:
    Pool() { for (std::size_t i = 0; i < N; ++i) slots_[i].nextFree = i + 1;
}

    template <typename ... Args> T* allocate(Args&& ... args) {
        if (freeHead_ == N) return nullptr;             // exhausted: the
caller decides
        Slot& s = slots_[freeHead_];
        freeHead_ = s.nextFree;                         // pop the free list
        return new (s.storage) T(std::forward<Args>(args) ... );   // placement
new: construct in place
    }
    void deallocate(T* p) {
        p->~T();
        auto* s = reinterpret_cast<Slot*>(p);
        s->nextFree = freeHead_;                        // push the slot back
        freeHead_ = s - slots_.data();
    }
};
```

Placement new constructs an object in memory that already exists, so no allocation happens. The program in `12-low-latency` replaces `operator new` with a counter and shows **0 heap allocations** on the pool path, 0 for a `pmr::vector` in an arena, and 1 for a `vector` with `reserve(100)`.

```
alignas(Task) unsigned char storage[sizeof(Task)];   // raw, correctly
aligned memory: no object yet
Task* o = new (storage) Task(7);                     // placement new:
construct INTO this memory, nothing is allocated
// ... use *o ...
o->~Task();                                           // you must call the
destructor yourself: never delete it
```

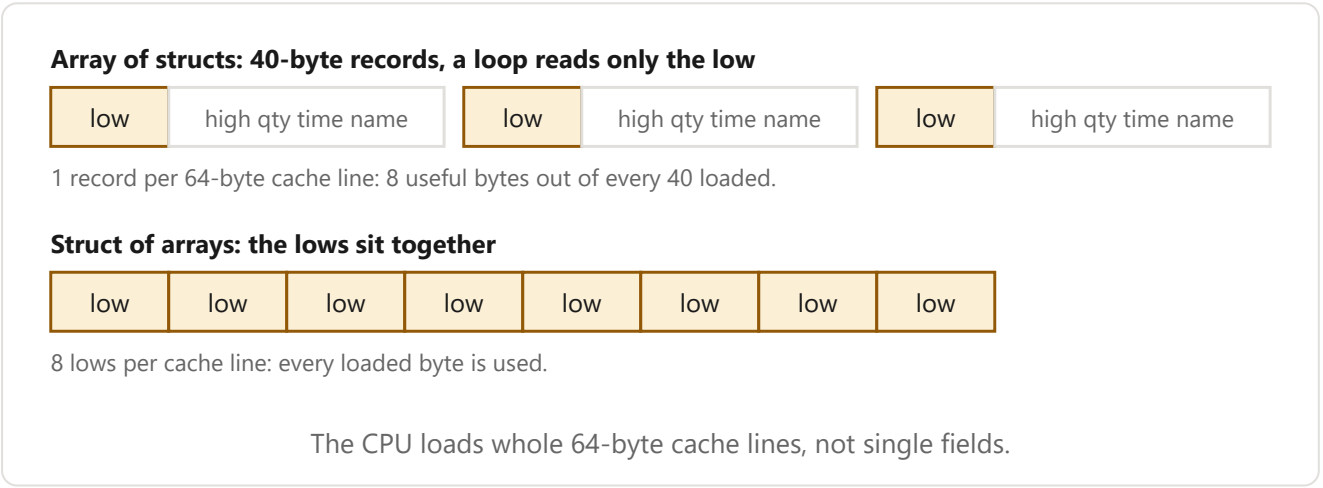
- The address of `o` equals the address of `storage` (compiled). The memory must be aligned and big enough.
- This is how `std::vector`, `std::optional` and every pool build objects in storage they already own. C++20 `std::construct_at` does the same without the `new` syntax.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 19.1 Controlling memory allocation (PDF pages 849 to 853)

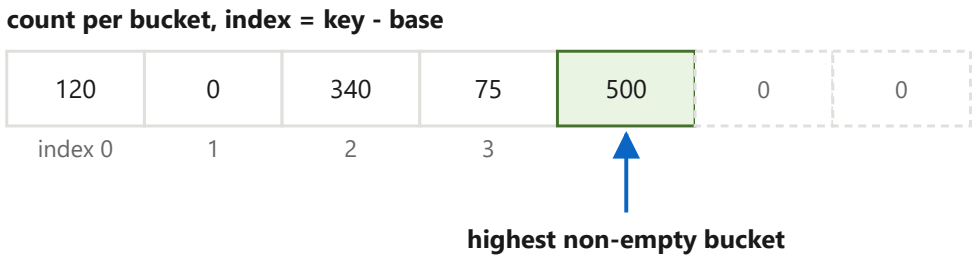
Cache-friendly data



- A **cache line** is 64 bytes. A cache miss to main memory costs about a hundred times a hit, so **locality** dominates: data used together should sit together.
- **Contiguous beats linked.** `std::vector` and `std::array` stream through memory; `std::list` and `std::map` chase pointers to nodes scattered across the heap (each node is also a separate allocation).
- **AoS versus SoA.** Array of structs keeps one record together (good when you use most fields); struct of arrays keeps each field together (good when a loop uses one or two fields). The program shows 40,000 bytes streamed to read all lows with AoS and 8,000 with SoA, with identical results.
- **Hot/cold split:** keep fields used on every message together and small; move rarely used fields elsewhere (struct size and padding: Binary Data and Parsing).

FALSE SHARING Two threads writing *different* variables on the *same* cache line make the line bounce between cores. Give each its own line: `alignas(64)` `std::atomic<size_t> head_; alignas(64) std::atomic<size_t> tail_;` (`sizeof` grows from 16 to 128 bytes, which is the point). Introduced in Multithreading.

Dense keys: an array instead of a map



Update = one array write. When the highest bucket empties, scan down to the next one.

A bounded key range turns a map into a flat array.

When the keys are **integers in a small, known range** (histogram buckets, status codes, small scores, sensor ids), a plain array indexed by `key - base` beats a `std::map` or a hash map. The operations are always the same: **update a bucket, find the highest non-empty one, look up an item by id**.

STRUCTURE	UPDATE	HIGHEST KEY	TRADE-OFFS
`std::map<key, count>`	O(log n)	O(1) (`begin()` or `rbegin()`)	simple and sorted, but a heap node per key (allocation, pointer chasing)
sorted `std::vector`	O(n) shift	O(1)	contiguous; fine when there are few keys
array indexed by key	**O(1)**	O(1) tracked, rescan on empty	no allocation, cache-friendly; needs a bounded key range and memory proportional to it
hash map `id` → `(key, count)`	O(1) average	n/a	for **lookup by id**; an open-addressing table avoids per-node allocation

```cpp
class BucketTable {
    static constexpr std::int64_t kBuckets = 4096;
    std::int64_t base_;                                  // key of index 0
    std::array<std::uint64_t, kBuckets> count_{};
    std::int64_t highest_ = -1;                          // -1 when empty
public:
    void add(std::int64_t key, std::uint64_t n) {
        auto i = key - base_;
        count_[i] += n;
        if (i > highest_) highest_ = i;                  // O(1)
    }
    void remove(std::int64_t key, std::uint64_t n) {
        auto i = key - base_;
        count_[i] = n ≥ count_[i] ? 0 : count_[i] - n;
        if (count_[i] == 0 && i == highest_)             // the highest
bucket vanished: scan down
            while (highest_ ≥ 0 && count_[highest_] == 0) --highest_;
    }
};
```

The program runs 200,000 random adds and removes against both this array and a `std::map`, and checks the highest key after every operation: 0 mismatches. A key outside the range needs a fallback (a re-centred range or an overflow structure).

Queues between threads

Pipeline stages run on separate threads and hand messages over a queue. With **one producer and one consumer** (SPSC) no lock is needed: each index is written by exactly one thread (Multithreading).

STEP THROUGH **An SPSC ring with 4 slots: the counters keep counting, the mask wraps them**

```
1  SpscRing<int, 4> ring;
2  push(10); push(20); push(30);
3  pop();
4  push(40); push(50);
5  push(60);
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 1 `SpscRing<int, 4> ring;`	An empty ring: **head** (next to read) and **tail** (next to write) are both 0. The slot index is `counter & 3`. The size is `tail - head`.	**`head = 0 (slot 0)`** | **`tail = 0 (slot 0)`** | **`slot0 = empty`** | **`slot1 = empty`** | **`slot2 = empty`** | **`slot3 = empty`**
2	line 2 `push(10); push(20); push(30);`	Three pushes write slots 0, 1 and 2, and only the **producer** moves **tail** (to 3). **head** is untouched: each index has exactly one writer, so no lock is needed.	`head = 0 (slot 0)` | **`tail = 3 (slot 3)`** | **`slot0 = 10`** | **`slot1 = 20`** | **`slot2 = 30`** | `slot3 = empty`
3	line 3 `pop();`	The **consumer** reads slot 0 (the value 10) and moves **head** to 1. That slot can now be reused. prints: `pop -> 10\n`	**`head = 1 (slot 1)`** | `tail = 3 (slot 3)` | **`slot0 = free (10 was read)`** | `slot1 = 20` | `slot2 = 30` | `slot3 = empty`
4	line 4 `push(40); push(50);`	40 goes to slot 3 (tail 3), then **tail = 4 and 4 & 3 = 0**: the index **wraps** to slot 0, where 50 lands on the freed slot. The counter went to 5 but the index is 1. The ring now holds 4 items: `tail - head = 4`.	`head = 1 (slot 1)` | **`tail = 5 (5 & 3 = slot 1)`** | **`slot0 = 50`** | `slot1 = 20` | `slot2 = 30` | **`slot3 = 40`**

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
5	line 5 `push(60);`	`tail - head == 4 == N`: the ring is **full** and `tryPush` returns false. It never grows and never overwrites unread data. What happens to 60 is a **policy**: drop it, drop the oldest, conflate, or (not in a network reader) wait. prints: `push 60 → refused (full)\n`	`head = 1 (slot 1) | tail = 5 (5 & 3 = slot 1) | slot0 = 50 | slot1 = 20 | slot2 = 30 | slot3 = 40`

```cpp
template <typename T, std::size_t N>
class SpscRing {
    static_assert((N & (N - 1)) == 0, "capacity must be a power of two");
    T buf_[N];
    alignas(64) std::atomic<std::size_t> head_{0};       // consumer writes
    alignas(64) std::atomic<std::size_t> tail_{0};       // producer writes
public:
    bool tryPush(const T& v) {
        auto t = tail_.load(std::memory_order_relaxed);
        if (t - head_.load(std::memory_order_acquire) == N) return false;
// full
        buf_[t & (N - 1)] = v;                           // mask instead of
%: the counters just keep counting up
        tail_.store(t + 1, std::memory_order_release);
        return true;
    }
    bool tryPop(T& out) {
        auto h = head_.load(std::memory_order_relaxed);
        if (h == tail_.load(std::memory_order_acquire)) return false;
// empty
        out = buf_[h & (N - 1)];
        head_.store(h + 1, std::memory_order_release);
        return true;
    }
};
```

- **Power-of-two capacity** turns the wrap-around into `index & (N - 1)`, one instruction, with free-running counters.
- **Padded indices** (`alignas(64)`) so the two threads do not fight over one cache line.
- **Always bounded.** An unbounded queue turns a slow consumer into growing memory and growing delay. When the queue is full you choose a **policy**:

POLICY	BEHAVIOR	USE WHEN
block (back-pressure)	the producer waits	nothing may be lost, and the producer can afford to wait (not for a network reader that must keep up)
drop newest	`tryPush` returns false, count it	simple; the consumer will need a recovery
drop oldest	overwrite the oldest entry	only the latest data matters
conflate	keep only the latest update per key	slow consumers of status updates

The program runs two real threads over a 1024-slot ring with a drop-newest policy: `received + dropped = 200000` and the consumer's sum equals the producer's, in 20 of 20 runs.

A predictable hot path

Besides memory and queues, a few language features add unpredictable cost. Keep them off the hot path:

AVOID ON THE HOT PATH	WHY	USE INSTEAD
exceptions (`throw`)	a throw costs microseconds and its cost varies; unwinding tables add size	error codes, `std::optional`, a small result struct; validate at the boundary
virtual calls	an indirect call: no inlining, harder for the CPU to predict	templates, CRTP, `std::variant` with `std::visit` (decide the type at compile time)
locks (`std::mutex`)	contention means waiting and context switches	single-writer data, SPSC queues, atomics
logging and I/O	system calls, formatting, locks inside the logger	push a small record onto a queue; a background thread formats and writes

`std::function`, `shared_ptr`	possible allocation, an indirect call, an atomic reference count	templates or function pointers, `unique_ptr` or plain ownership
`std::map`, `std::list`, `std::string` building	allocation per node or per string	arrays, pools, fixed buffers, `string_view`

```
enum class ParseError { None, Truncated, BadType, BadLength };

ParseError parseSample(const unsigned char* buf, std::size_t len, Sample&
out) noexcept {
    if (len < kSampleSize) return ParseError::Truncated;     // no exception:
a value the caller checks
    if (buf[0] ≠ 'T')     return ParseError::BadType;
    // ... fill `out` ...
    return ParseError::None;
}
```

- Mark functions that cannot throw `noexcept`; it lets the compiler skip unwinding code.
- `constexpr` lookup tables move work to compile time; `[[likely]]` / `[[unlikely]]` (C++20) hint at the common path.
- These rules are for the **hot path**. Startup, configuration and error reporting can use exceptions and heap freely.

Threads, cores and busy-polling

- **Pin each thread to a core** (CPU affinity). The OS stops migrating it, so its caches stay warm and its timing steady. Give hot threads their own cores, and do not run more busy threads than cores (**oversubscription** causes context switches).
- **Busy-poll instead of blocking.** A thread that sleeps on a condition variable pays a wake-up delay (microseconds) when work arrives. A thread that spins checking a flag reacts almost instantly but **uses 100% of its core**. Latency-critical threads spin; others block.
- **Memory placement:** on multi-socket machines keep a thread and its data on the same NUMA node. Pre-touch (page-fault in) buffers at startup so the first use on the hot path does not take a page fault.
- Isolating cores from the scheduler and tuning interrupts is an operating-system job, usually done by the platform team.

Locks on the hot path

PROBLEM	WHAT HAPPENS	REMEDY
Lock contention	threads queue on one mutex, so the program is effectively serial (Amdahl)	shorter critical sections, shard the data, per-thread data merged at the end
Priority inversion	a low-priority thread holds a lock a high-priority one needs, and a medium one keeps the low one from running	priority inheritance, shorter locks, no locks shared across priorities
Context switches	blocking and waking a thread costs microseconds and cold caches	brief spinning, pin threads to cores, keep waits off the hot path
Allocation under contention	`malloc` may lock; latency is unpredictable	reserve, pools and ring buffers

```
// busy-poll: spin on an atomic instead of sleeping (burns the core, minimal
wake-up delay)
while (!ready.load(std::memory_order_acquire)) {
    _mm_pause();                                          // x86 hint: be nice
to the sibling hyperthread
}

// Linux only: pin the calling thread to core 3
cpu_set_t set;
CPU_ZERO(&set);
CPU_SET(3, &set);
pthread_setaffinity_np(pthread_self(), sizeof(set), &set);
```

KEY POINT "How would you design a program that processes a fast stream of messages?" Reader thread pinned to a core, reading the socket and parsing with zero copy into preallocated structures; an SPSC queue to a processing thread, which updates array-based tables and publishes; no allocation, locks, exceptions or logging on the path; bounded queues with a stated drop or recovery policy; sequence-number gap detection; and p99.9 latency measured under realistic load.

READ MORE

C++ Concurrency in Action (Williams)

- 8.2 Factors affecting the performance of concurrent code (cache ping-pong, false sharing, oversubscription) (PDF pages 283 to 289)
- 8.3 Designing data structures for multithreaded performance (PDF pages 289 to 292)

Cheat sheet

Answer aloud, then open.

<details>
<summary>▼ Why does tail latency matter more than the average in a message pipeline?</summary>

The rare slow case (p99.9) is what produces a stale result or a missed deadline, and the average hides it: a mean of 281 ns can coexist with a p99.9 of 80 microseconds. Measure and report percentiles, and remove sources of unpredictable delay.
</details>

<details>
<summary>▼ Why avoid <code>new</code> on the hot path, and what do you do instead?</summary>

The allocator can take a lock, walk free lists, fragment and call the kernel, so its cost is unpredictable. Allocate at startup: `reserve` containers, use an object pool or an arena (`std::pmr::monotonic_buffer_resource`), and use fixed buffers instead of strings and vectors for small data.
</details>

<details>
<summary>▼ How does an object pool work?</summary>

A fixed array of slots, plus a free list threaded through the free slots. Allocation pops the head of the list and constructs the object there with placement new; deallocation destroys it and pushes the slot back. Both are a few instructions with no heap, and when the pool is empty the caller decides what to do.
</details>

▼ Array of structs versus struct of arrays?

AoS keeps one record's fields together, good when you use most fields. SoA keeps each field in its own array, good when a loop uses one or two fields: it streams only the bytes it needs, so each cache line is fully useful. Same results, less memory traffic.

▼ What is false sharing and how do you fix it?

Two threads writing different variables on the same 64-byte cache line force the line to bounce between cores. Give each variable its own line with `alignas(64)` or padding, as for the head and tail indices of an SPSC queue.

▼ Why is a vector or array usually faster than `std::map` or `std::list`?

Contiguous memory is cache-friendly and prefetchable; node-based containers chase pointers across the heap and allocate per node. Big-O hides these constants, especially for small and medium sizes.

▼ When would you use an array indexed by key instead of a map?

When the keys are integers in a bounded range: updates are O(1) with no allocation and the data is contiguous, so it is cache-friendly. Track the highest or lowest non-empty bucket and rescan when it empties. A hash map from id to its bucket handles lookups by id. A `std::map` is simpler but allocates a node per key.

▼ Why is an SPSC ring capacity a power of two?

The wrap-around becomes `counter & (N - 1)` instead of a modulo, one cheap instruction, and the head and tail counters can simply keep counting up while `tail - head` is the size.

▼ What do you do when a queue between threads is full?

Never let it grow without bound. Choose a policy: block (back-pressure), drop the newest, drop the oldest, or conflate to the latest per key, count what was dropped, and recover from the gap if needed. A network reader normally cannot block.

▼ Why avoid exceptions, virtual calls, locks and logging on the hot path?

Each adds cost that is large or unpredictable: a throw costs microseconds, a virtual call blocks inlining and is an indirect branch, a mutex can wait and context-switch, and logging means system calls and formatting. Use error codes, templates or CRTP, single-writer data, and an asynchronous logger fed by a queue.

▼ Busy-polling versus blocking, and why pin threads?

Blocking saves CPU but a wake-up costs microseconds; busy-polling reacts almost immediately but burns a whole core, so only latency-critical threads spin. Pinning a thread to a core stops migration, which keeps its caches warm and its timing stable.